aimshalf

Installation Guide

Installing aimshalf, the DFT-1/2 band-gap correction for FHI-aims, from scratch

Version 0.3.0 · October 2026

Eduardo Afonso Moreira Santos — IFSC-USP · advisor Guilherme Matos Sipahi

License: GPL-3.0 (includes components of minushalf, GMSN/ITA)

Contents

1. What you are going to install . . . 3

1.1 Requirements . . . 3

2. Get the source code . . . 3

3. Install the Python package . . . 4

4. Build the atomic solver . . . 4

5. Check the Python side . . . 4

6. Add the aimshalf hooks to FHI-aims and build it . . . 5

6.1 Install the hooks . . . 5

6.2 Build . . . 5

6.3 What the hooks change . . . 6

7. Verify the complete installation . . . 6

8. Optional: validation against minushalf . . . 6

9. Using aimshalf on a cluster . . . 7

10. Updating and uninstalling . . . 7

11. Troubleshooting . . . 8

12. Getting help . . . 8

1. What you are going to install

aimshalf brings the DFT-1/2 band-gap correction (LDA-1/2 / GGA-1/2, Ferreira, Marques & Teles 2008) to the all-electron code FHI-aims. It follows the workflow of minushalf (the GMSN/ITA implementation for VASP). A working installation has three parts:

Part	What it is	Where it lives
1. Python package	The **aimshalf** command: builds the self-energy potentials, runs FHI-aims, optimizes r_{cut}, reads results.	`aims_half/aimshalf/`
2. Atomic solver	L. G. Ferreira's modified ATOM program (from minushalf), compiled once into `atom/atmhalf`. It computes the neutral and half-ionized atoms.	`aims_half/atom/`
3. FHI-aims with the aimshalf hooks	Your own FHI-aims build plus short hooks (added by an installer) for the keyword `aimshalf_potential` (an atom-centred potential added to the Kohn–Sham potential).	your FHI-aims source tree

> **Note.** The FHI-aims source code is not distributed with aimshalf. You need your own access to FHI-aims (a licensed source tarball of release 261002, or 260331.1).

1.1 Requirements

Component	Needed for	Check with
Linux (x86-64)	everything	`uname -a`
Python ≥ 3.9 with venv	part 1	`python3 --version` `python3 -m venv --help`
git	getting the source	`git --version`
gfortran + make	part 2	`gfortran --version` `make --version`
FHI-aims build tool chain: CMake, Fortran/C compilers, MPI, BLAS/LAPACK/ScaLAPACK	part 3	`cmake --version` `mpif90 --version`
About 2 GB of disk	the FHI-aims build	`df -h ~`

On Ubuntu/Debian the system tools can be installed with:

```
sudo apt install python3-venv python3-full git gfortran make cmake \
                 openmpi-bin libopenmpi-dev libscalapack-openmpi-dev \
                 liblapack-dev libblas-dev
```

On a cluster, load the equivalent modules instead (for example `module load gcc openmpi cmake`); ask your system administrator which ones FHI-aims uses there.

2. Get the source code

aimshalf is distributed as a git repository. Use whichever of the following you received:

From a repository URL or a shared folder

```
cd ~/software                    # any folder you like
git clone <URL-or-path>/aims_half aims_half
cd aims_half
git checkout v0.3.0              # the released version
```

From a single bundle file (aimshalf-0.3.0.bundle)

```
git clone aimshalf-0.3.0.bundle aims_half
cd aims_half && git checkout v0.3.0
```

You should now see this layout:

```
aims_half/
├── aimshalf/       Python package
├── atom/           ATOM solver source + atom/fixtures/<Element>/INP
├── fhiaims/        FHI-aims hook installer, module source, install tests
├── examples/       annotated aimshalf.yaml
├── scripts/        plotting scripts
├── tests/          automated tests
├── validation/     comparison with minushalf
├── README.md  CHANGELOG.md  LICENSE  pyproject.toml  Makefile
```

Important. Keep this folder: aimshalf reads the atomic solver and the per-element inputs from it at run time. Do not delete it after installing.

3. Install the Python package

Install aimshalf into its own virtual environment. Run these commands **from the aims_half folder**:

```
cd ~/software/aims_half
python3 -m venv .venv
source .venv/bin/activate          # the prompt now starts with (.venv)
pip install -e ".[dev]"            # aimshalf + numpy, pyyaml, pytest, matplotlib
aimshalf --version                 # -> aimshalf 0.3.0
```

- **-e** (editable mode) is required: the package finds `atom/atmhalf` and `atom/fixtures/` in the source folder.
- **[dev]** adds pytest and matplotlib (tests, plotting scripts, validation). For a minimal install use `pip install -e .`.
- Activate the environment in every new terminal (`source .venv/bin/activate`), or put that line in your job scripts.

Note. If you must install without -e (for example into a shared environment), set `export AIMSHALF_ROOT=/path/to/aims_half` so that aimshalf can find the atomic solver and its inputs.

4. Build the atomic solver

```
make atom
ls -l atom/atmhalf                 # the executable must exist
```

This runs `gfortran -std=legacy -w -O2` on `atom/driver.f atm_cGuima3.f` and takes a few seconds. It needs to be done once per machine (and again only if you change compiler or move to another architecture).

5. Check the Python side

```
python -m pytest tests/ -q          # expected: 20 passed
aimshalf gen-sep As -c p=0.5 --cut 3.8 --out-dir /tmp/sep_test
```

The second command should print:

```
SEP written: /tmp/sep_test/sep_As_cut3.800.dat
tail-charge check: fitted q = 0.4383 e (expected ~ 0.5000)
```

A fitted charge somewhat below 0.5 e is normal: the fit is made at 6–12 bohr, where the PBE tail has not yet reached the full −q/r form.

> **Note.** If two tests are reported as *skipped*, the atomic solver was not built (section 4).

6. Add the aimshalf hooks to FHI-aims and build it

aimshalf does not distribute any FHI-aims code. A small installer adds the aimshalf lines to **your own** FHI-aims source tree (one Fortran module and short hooks in five files). Use a **pristine** (unmodified) tree; tested with releases 261002 (recommended) and 260331.1.

6.1 Install the hooks

```
cd ~/software
tar xzf fhi-aims.261002.tar.gz
python3 ~/software/aims_half/fhiaims/install_hooks.py ~/software/fhi-aims.261002 --check
python3 ~/software/aims_half/fhiaims/install_hooks.py ~/software/fhi-aims.261002
```

The installer locates every insertion point before it writes anything, and reports:

```
FHI-aims source: /home/you/software/fhi-aims.261002/src
  + CMakeLists.txt: module in library aims4 (line 622)
  + dimensions.f90: keyword accepted in the first pass (line 1654)
  + read_control.f90: use statement (line 118)
  + read_control.f90: declarations (line 392)
  + read_control.f90: keyword aimshalf_potential (line 3258)
  + prepare_scf.f90: use statement (line 41)
  + prepare_scf.f90: read potential files, map species (line 343)
  + integrate_hamiltonian_matrix_p2.f90: use statement (line 107)
  + integrate_hamiltonian_matrix_p2.f90: declaration (line 183)
  + integrate_hamiltonian_matrix_p2.f90: SEP added to the local potential (line 1434)
  + src/external_atomic_potential.f90 copied from aimshalf
aimshalf hooks installed. Now build FHI-aims as usual.
```

- `--check`: dry run, writes nothing.
- `--remove`: removes the hooks and the module; the files become identical to the original ones.
- Running it twice does not insert twice. If an insertion point is not found (another release) or the tree was already edited, it stops and changes nothing.

> **Important.** Start from a fresh copy of the tarball if the installer reports that the tree was already modified. To support another FHI-aims release, follow `fhiaims/INTEGRATION.md`.

6.2 Build

Build exactly as you normally build FHI-aims (see the FHI-aims manual). A typical CMake build looks like:

```
cd ~/software/fhi-aims.261002
mkdir build && cd build
cp ../initial_cache.example.cmake initial_cache.cmake   # then edit compilers/libraries
cmake -C initial_cache.cmake ..
make -j 8
ls ../build/aims.261002.scalapack.mpi.x   # name depends on your build options
```

Write down the full path of the executable: it goes into `aims_cmd` in every `aimshalf.yaml`.

6.3 What the hooks change

File	Change
`CMakeLists.txt`	adds the module to library aims4
`dimensions.f90`	accepts the new keyword in the first pass over control.in
`read_control.f90`	reads `aimshalf_potential <species> <file>`
`prepare_scf.f90`	reads the potential files once, builds splines
`integrate_hamiltonian_matrix_p2.f90`	adds V_S to the local potential at every grid point, every SCF iteration

> **Important.** The keyword is `aimshalf_potential` since version 0.3.0 (the old name `external_atomic_potential` is still accepted). A binary built with an older aimshalf version does not know the new name: rebuild it. `aimshalf execute` checks this before it starts.

Without the keyword in control.in, the modified binary behaves exactly like the stock one. Total energies with the potential switched on are not meaningful (DFT-1/2 corrects eigenvalues only).

7. Verify the complete installation

The install tests run a small GaAs calculation three times (plain PBE, zero potential, As-p self-energy potential). About one minute per run on 4 cores.

```
cd ~/software/aims_half
source .venv/bin/activate
export AIMS_PATCHED=~/software/fhi-aims.261002/build/aims.261002.scalapack.mpi.x
export AIMS_SPECIES=~/software/fhi-aims.261002/species_defaults/defaults_2020/light
export MPI_LAUNCH="mpirun -np 4"
bash fhiaims/run_install_tests.sh
```

Expected result (from FHI-aims 261002):

```
  [PASS] aimshalf on PATH: .../.venv/bin/aimshalf
  [PASS] gen-sep As p=0.5 cut=3.8 ran
  ...
  [PASS] patched aims recognised the keyword
  [PASS] zero SEP leaves total energy unchanged (-115076.886712548 eV)
        PBE gap = 0.53720552 eV ; PBE-1/2 (As p, cut 3.8) gap = 1.59651396 eV
  [PASS] SEP opens the gap by more than 0.3 eV

RESULT: 10 passed, 0 failed
```

- Optional: set `AIMS_STOCK` to an unpatched binary of the same release to also check that patched and stock aims give identical energies without the keyword.
- The tests write into `aims_half/install_tests/` (ignored by git).

8. Optional: validation against minushalf

This compares aimshalf's self-energy potentials with minushalf's own code (22 checks). It needs the minushalf source folder next to `aims_half` (or its path in `MINUSHALF_ROOT`).

```
cd ~/software
ls                       # aims_half  minushalf
cd aims_half && source .venv/bin/activate
python validation/compare_minushalf.py 2>&1 | tail -3
# SUMMARY: 22/22 checks passed
```

Plots and the report are written to `validation/results/`.

9. Using aimshalf on a cluster

- Install as above on the cluster login node (venv, `make atom`, FHI-aims with the aimshalf hooks built with the cluster tool chain).
- Inside a SLURM, PBS or LSF job, `aimshalf execute` runs in the foreground automatically; outside a job it detaches into the background.
- Use the cluster launcher in `aims_cmd`, e.g. `srun aims.x` or `mpirun -np 64 /path/aims.x`.

Example SLURM job script

```
#!/bin/bash
#SBATCH --job-name=gaas-half
#SBATCH --nodes=1
#SBATCH --ntasks=32
#SBATCH --time=24:00:00
module load gcc openmpi              # as used to build aims
source ~/software/aims_half/.venv/bin/activate
cd $SLURM_SUBMIT_DIR                 # folder with control.in, geometry.in, aimshalf.yaml
aimshalf execute
```

with `aims_cmd: "srun /path/to/aims.261002.scalapack.mpi.x"` in aimshalf.yaml. If the job is killed, submit it again: finished runs are reused.

10. Updating and uninstalling

```
cd ~/software/aims_half
git fetch && git checkout v0.X.Y         # or: git pull
source .venv/bin/activate
pip install -e ".[dev]"                  # only needed if dependencies changed
python -m pytest tests/ -q
```

Read `CHANGELOG.md` after updating: version 0.3.0, for example, changed the default gap from fundamental to smallest direct gap (option `indirect`).

To uninstall: `pip uninstall aimshalf`, then delete the `aims_half` folder. The FHI-aims build is independent and can be kept or deleted (or restored with `install_hooks.py --remove`).

11. Troubleshooting

Message	Cause and fix
`aimshalf: command not found`	The virtual environment is not active. Run `source ~/software/aims_half/.venv/bin/activate`.
`error: externally-managed-environment`	pip was run outside the venv (Ubuntu protects the system Python). Activate the venv first.
`bash: .venv/bin/activate: No such file`	You are not in the aims_half folder. `cd` there first.
`No INP fixture for species X`	Installed without -e. Reinstall with `pip install -e .` or set `AIMSHALF_ROOT`.
`atomic solver not built`	Run `make atom` in aims_half (needs gfortran).
`no aims executable found in aims_cmd`	Set `aims_cmd` in aimshalf.yaml to the full path of the FHI-aims binary built with the aimshalf hooks.
`control.in has no 'output band ...' lines`	Add band-path lines; aimshalf reads the gap and the band-edge character on the band path.
`install_hooks: ... not found`	The FHI-aims release is not supported yet, or the path is not the root of the source tree (it must contain src/).
`install_hooks: ... already modified`	The tree was edited before (older aimshalf patch or by hand). Start from a fresh tarball.
aims stops: `cannot open file`	The potential file named in control.in is missing; let aimshalf write control.in (do not copy it by hand from a run folder).
keyword not recognized by aims	The binary in `aims_cmd` was built without the aimshalf hooks.
`does not know the keyword 'aimshalf_potential'`	FHI-aims was built with an older aimshalf version (or without the hooks). Install the current hooks and rebuild (section 6).
`ModuleNotFoundError: matplotlib`	Install the extras: `pip install -e ".[dev]"`.
install tests: many FAIL after the first	Usually the venv was not active; the script now stops at the first Python failure.

12. Getting help

Run `aimshalf -h` and `aimshalf <command> -h`. The User Guide describes every command and option. Contact: Eduardo Afonso Moreira Santos, IFSC-USP (duardo1510@ifsc.usp.br).